
CommerceCore Expansion: Auditing Small-Model Adaptation for Product Matching and Attribute Extraction

Arghya Mukherjee
Independent Researcher
arghya05@gmail.com
ORCID: 0009-0008-3423-8574

Abstract

Small language models could support several product-understanding operations within a retailer’s infrastructure, but evidence of task accuracy is easily confounded with evidence of generalization or serving efficiency. We study QLoRA adaptation of Qwen3-1.7B for query–product relevance, listing identity, functional relation, technical compatibility, and brand/color extraction. We reconstruct the experiments from committed data, checkpoint histories, training logs, prediction records, and serving traces, and release an offline audit that regenerates the quantitative analysis. On a common 200-listing extraction sample, the dedicated adapter attains brand F1 of 1.000 and color F1 of 0.955 under the recorded permissive matching rule; the latter exceeds the strongest tested API comparator by 1.38 percentage points. This is a descriptive result: two evaluation texts also occur in training, and missing paired outputs prevent a significance claim. Matching results expose a different problem. Adapter and API relevance scores use different samples; a single-token relevance adapter reaches 0.5115 accuracy but only 0.436 macro-F1; and 25 of 28 functional-relation test rows reuse training scenarios. A checkpoint sweep further exposes that test partition to model selection. We formalize the information required for comparable evaluations, demonstrate scorer counterexamples, and bound the paired correctness uncertainty recoverable from aggregate extraction counts. Available load tests contain no completed inference requests and therefore establish neither throughput nor cost advantage. The evidence supports promising narrow extraction behavior and a reproducible account of evaluation failure modes, rather than universal superiority or a production-ready horizontal model.

1 Introduction

Product understanding is a collection of decisions with different operational consequences. Search relevance asks whether an offer satisfies a query. Entity matching asks whether two offers identify the same purchasable item. Functional relation asks whether products substitute for or complement one another. Technical compatibility asks whether a particular interface or device revision fits. Attribute extraction converts listing text into structured catalog fields. An inexpensive model covering these decisions would be useful in catalog ingestion, search enrichment, and offer consolidation, especially for organizations that cannot maintain many large models.

These tasks cannot safely share semantics merely because they share a language model. Two distinct phones can both be exact matches for a query; an accessory can complement a device without fitting its revision; and a plausible attribute can be absent from the listing evidence. A credible evaluation

must preserve these distinctions while controlling the sample, the evidence shown to the model, the scoring rule, and the decisions made after examining evaluation results.

CommerceCore Expansion investigates this setting through a shared four-task adapter and separate adapters for extraction and functional relation. The study began as a sequence of practical adaptation experiments. The present manuscript adds a retrospective, executable audit of the resulting artifacts. This distinction matters: the experiments were not prospectively designed as a factorial study, and the audit does not turn configuration changes into causal ablations. Its value is to establish which conclusions remain supported when experimental provenance is treated as part of the result.

We address three research questions. **RQ1:** what task behavior is supported by the saved shared and specialist adaptation experiments? **RQ2:** which model comparisons remain interpretable after checking split independence, input equivalence, scoring, and checkpoint selection? **RQ3:** what can the recorded training and serving measurements establish about efficient deployment? These questions connect the empirical model study to a practical problem in information retrieval: deciding when an offline product-understanding score is strong enough to justify a system claim.

Contributions. First, we document an implemented small-model system spanning five task formulations and reconstruct its data mixture, training configuration, and complete available checkpoint trajectories. Second, we release an executable comparison audit with dataset-overlap checks, class-level error reconstruction, scorer counterexamples, and source hashes. Third, we analyze a matched extraction result and derive the range of paired correctness tests consistent with its aggregate counts, making explicit what missing predictions prevent us from concluding. Fourth, we report negative and incomplete evidence, including minority-class errors, scenario reuse, test-based checkpoint selection, and censored load tests. These are empirical and reproducibility contributions; the work does not introduce QLoRA, claim a new foundation-model architecture, or establish state of the art on a public leaderboard.

Artifact boundary. The code and evidence are available at <https://github.com/arghya05/commercecore-expansion>. The audit starts from the source tree at public commit 2af4b2c (originally observed locally as c7deee7, with an identical tree) and supplements it with archived local trainer states and adapter configurations. It does not rerun training or paid APIs. This project is separate from the earlier CommerceCore query-to-constraints parser [1]: the experiments load the upstream Qwen base rather than that release’s merged model. No result here establishes a quality comparison with the earlier parser, whose task is different.

2 Related Work and Research Position

Commerce instruction tuning. eCeLLM [2] studies commerce instruction tuning across multiple tasks and generalization settings, including held-out task and category conditions. Its generalist/specialist and data-size analyses are closer to controlled evidence of transfer than the sequential configurations available here. Our local eCeLLM-S evaluations are therefore treated as implementation-specific measurements, with prompt and parser limitations, rather than a reproduction of its published benchmark. EcomGPT [3] develops commerce instruction data through task transformations and chain-of-task construction. Both works establish that broad commerce instruction tuning is prior art; combining several retail labels in one adapter is insufficient by itself to establish methodological novelty.

Multimodal commerce. CASLIE [4] uses context-sensitive image captions and caption-quality selection to construct commerce instruction inputs. Its multimodal evidence and generalization experiments differ from our text-only product-pair and extraction setting. We do not infer superiority over CASLIE from a text-only table, and the present audit does not evaluate visual attributes, image consistency, or multimodal retrieval.

Small models and deployment trade-offs. Licardo and Tanković [5] are particularly close in deployment motivation: they adapt a small Llama model on synthetic commerce instructions and examine optimization choices, including quantization and hardware behavior. Their task centers on structured intent/action outputs; ours separates product relevance, identity, relations, and extraction.

Table 1: Research position. “Not measured” denotes a missing experiment, not an architectural inability of a comparator.

Prior work	Relevant overlap	Boundary of this study
eCeLLM / EcomGPT CASLIE	Commerce instruction tuning across tasks Commerce data quality and generalization	No matched reproduction of their full suites Images and multimodal transfer not measured
Licardo– Tanković Ditto / WDC Products	Small-model adaptation and deployment Product entity matching	No measured quantization or energy trade-off No supervised Ditto baseline or entity-disjoint sweep
GLiNER2 This study	Compact structured extraction Shared/specialist QLoRA and artifact audit	Local two-field comparison only Evidence validity and narrow extraction signal

Table 2: Implemented task formulations. “Extraction” contains two evaluated fields.

Task	Input	Observed output labels
Relevance	Query and product listing	exact, substitute, complement, irrelevant
Identity	Two product listings	same, distinct
Functional relation	Two product descriptions	substitute, complement, unrelated
Compatibility	Two product descriptions	compatible, incompatible
Extraction	Listing text	JSON fields: brand, color

Their measured deployment comparisons also identify a missing component of this study: parameter count and QLoRA training alone do not establish low inference latency, energy use, or serving cost.

Matching and extraction alternatives. ESCI [6] supplies product-search relevance judgments, while WDC Products [7] explicitly varies entity-matching difficulty and unseen-entity conditions. Our subsets are derived from these sources but do not implement their complete benchmark protocols. Ditto [8] is a directly relevant supervised entity-matching baseline that remains untested here. GLiNER2 [9] provides schema-driven extraction and classification with an encoder architecture, illustrating why a generative model must be compared with compact task-appropriate systems, not only general-purpose APIs. Our GLiNER measurements concern two fields on one ABO-derived sample [10]; they do not rank the systems across their broader capabilities.

Adaptation and evaluation methodology. LoRA [11] and QLoRA [12] supply the adaptation mechanism used here. Gradient balancing and gradient-conflict methods such as GradNorm [13] and PCGrad [14] are relevant prospective controls for multitask interference, but neither was implemented in these runs. Data leakage is a recognized source of irreproducibility [15], and behavioral testing frameworks such as CheckList [16] motivate testing a scorer on deliberately constructed cases. Our contribution is an application of these principles to a concrete commerce adaptation record, including newly computed overlap statistics and uncertainty bounds, rather than a claim to have invented leakage prevention or behavioral testing.

3 Tasks, Data Construction, and Split Integrity

3.1 Operational task definitions

Table 2 defines the labels actually represented in the experimental data. The broader project schemas include an *unknown* state for some relations, but these experiments do not train or evaluate a calibrated abstention class. We therefore do not count schema support as measured model capability. Functional context and compatibility constraints can be present in the scenario metadata without being fully exposed in the generated listing prompt, an important limitation for tasks whose answer depends on such evidence.

Table 3: Current shared mixture and preserved v1 development split. The last column counts labels in the historical split, exposing the single-class minority-task evaluations. Counts are regenerated from row files.

Task	Current train	Current dev	Saved v1 dev	Saved labels
Relevance	8000	1000	1000	4
Identity	6000	3500	3500	2
Functional relation	52	13	15	1
Compatibility	32	8	4	1

3.2 Real-data mixtures and historical split versions

The current shared-adapter mixture contains 18605 rows, of which 14084 are assigned to training and 4521 to development. These counts are not interchangeable with the number of optimizer examples processed. Training may repeat or oversample rows, while the files also contain development data. Relevance uses ESCI-derived query-product pairs; identity uses WDC-derived listing pairs. Table 3 distinguishes the current files from the development rows saved with the original shared adapter.

The scaled v3 mixture contains 46,086 training and 7,519 development rows. Its relevance component grows to 40,000/4,000 train/dev rows; identity remains 6,000/3,500. It retains an earlier minority-task assignment of 50/15 functional-relation and 36/4 compatibility rows. Consequently, the current mixture is not a frozen representation of every historical run. In both relevance mixtures, the audit finds zero shared query identifiers and zero normalized full-input duplicates between train and dev. This is a positive split check, but it does not establish disjoint product families, merchants, or pretrained-model exposure.

The API relevance sample contains 60 rows, balanced at 15 per class; its identity sample contains 50 rows, balanced at 25 per class. None of their normalized formatted inputs exactly matches the current shared training or development inputs. This check establishes that those API samples differ from the adapter’s recorded development populations; it does not establish their independence at every product or query level. The shared identity development set is imbalanced: 3,000 of 3,500 labels are *distinct*, giving an always-distinct accuracy of 0.857. We report this reference because raw accuracy alone can conceal weak performance on true matches.

3.3 Scenario-conditioned synthetic data

The synthetic pipeline first defines a structured product-pair scenario and an intended relation. Claude Haiku generates surface descriptions from that scenario; GPT-5-mini subsequently assesses support for the assigned label. The judge sees the scenario and proposed label. This provides a cross-model consistency check, not blinded human adjudication, a factuality guarantee, or a measured naturalness score. Hard-coding the intended label constrains generation but does not guarantee that the generated text entails it.

The original functional set has 65 rows and the compatibility set 40. The expanded functional generation pass audits 124 rows and retains 122, which are merged with the original 65 to form 187 rows over 63 distinct scenario objects. The final row assignments are 131 train, 28 dev, and 28 named `locked_test`. All three functional labels occur in each final partition. However, stratification is performed over paraphrases rather than independent scenarios: 26/28 development and 25/28 test rows have a scenario object also present in training. There are no exact normalized generated-text duplicates across those partitions, so text deduplication alone would miss this dependence.

The earlier split repair creates an additional problem for a frozen model. Comparing the corrected development rows with the earlier training assignment preserved in the scaled mixture shows that 11/13 functional and 6/8 compatibility development texts were assigned to training under the earlier split. This is reconstructed exposure evidence, not a newly independent holdout. Re-stratifying existing examples cannot remove information already available during an earlier fit. These sets are useful diagnostics of label coverage; they do not support claims about unseen scenarios.

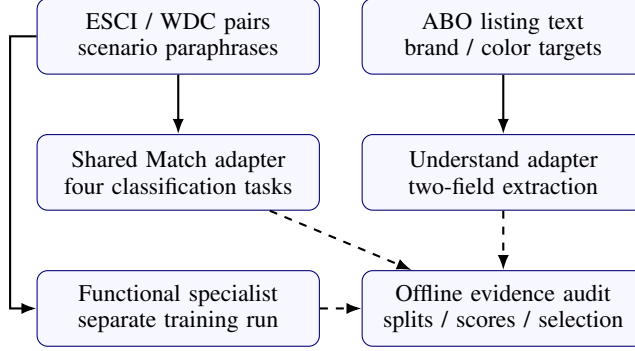


Figure 1: Implemented training paths and retrospective audit. Solid arrows denote data-to-adapter paths; dashed arrows denote evidence inspection. All adapters use the same upstream base family. The serving implementation loads separate model instances; this diagram does not imply shared resident base weights or an adaptive training controller. The experimental relevance-letter adapter is analyzed separately.

3.4 Attribute extraction and evidence grounding

The ABO-derived extraction files contain 9,396 training, 300 development, and 200 evaluation rows. Training has 9,359 unique item identifiers and 9,319 unique texts. Train/dev overlap comprises four item identifiers and two exact texts; train/evaluation overlap has zero item identifiers but two exact texts. An identifier-only exclusion rule therefore does not establish text independence.

All brand values in the three files occur literally in their listing text under case-insensitive matching. Color values do so for 8,390/9,396 training, 256/300 development, and 184/200 evaluation rows. Nonliteral colors may reflect normalization or metadata-specific names rather than necessarily incorrect labels. Nevertheless, these observations contradict a stronger claim that every gold value is an exact span of the supplied text. A strictly grounded benchmark would separate extraction, normalization, and inference, then adjudicate each case under the corresponding policy.

4 Adaptation Methods and Experimental Record

4.1 Shared and specialist adapters

All implemented adapters use the upstream Qwen/Qwen3-1.7B identifier. A shared Match adapter predicts four relation types using task-specific instructions. The Understand adapter predicts brand/color JSON; a dedicated functional adapter is trained after the shared runs. A separate single-token relevance adapter is retained as an experimental alternative. Thus the delivered system is a family of adapters, not evidence that one parameter state performs every commerce task well. Figure 1 distinguishes the training and serving roles.

The scripts quantize base weights with 4-bit NF4 and double quantization, using bfloat16 computation. Following LoRA [11], an adapted projection is

$$W' = W_{\text{base}} + \frac{\alpha}{r}BA, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}. \quad (1)$$

The frozen quantized base and trainable low-rank matrices follow the QLoRA setup [12]. The target modules are the query, key, value, and output attention projections and the gate, up, and down feed-forward projections. Archived specialist and v1 configurations use rank 16, scale 32, and dropout 0.05. The current shared script uses rank 32 and scale 64 for the v3 configuration; it also contains relevance oversampling. Running that script today does not reproduce the earlier v1 experiment unchanged.

The registry records a base revision, but the actual `from_pretrained` calls omit the revision argument and archived adapter configurations contain a null revision. We therefore record the intended pin separately from the executed loader behavior. Neither a registry entry nor a model name alone certifies the exact weights downloaded in every historical run.

Table 4: Recorded configurations. Learning rate, length, and batching describe preserved scripts; step counts are supported by histories/logs. Shared v1/v2 scripts were edited in place, so this table is not a fully frozen historical environment. Effective batch is per-device batch times accumulation on one device.

Run	Rank	Steps	Max length	Batch	LR	Objective
Shared v1	16	400	256	16	10^{-4}	Completion CE
Shared v2	16	800	256	16	10^{-4}	Resampled mixture
Shared v3	32	1,200	256	16	10^{-4}	Expanded mixture
Relevance letters	16	1,200	256	16	$3 \cdot 10^{-5}$	Token-weighted CE
Functional specialist	16	600	320	16	$2 \cdot 10^{-5}$	Token-weighted CE
Extraction specialist	16	1,200	384	16	10^{-4}	Completion CE

4.2 Training objective, batching, and label representation

Each example concatenates an instruction/input prefix x with a short target y and an end-of-sequence token. Prefix and padding labels are masked. The unweighted causal objective is

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{\sum_i |y_i|} \sum_i \sum_{j=1}^{|y_i|} \log p_{\theta}(y_{ij} \mid x_i, y_{i,<j}), \quad (2)$$

where only unmasked completion positions contribute. The implementations truncate the concatenated sequence from the right. They do not archive an audit of the fraction of targets removed by truncation; future controlled runs should reject examples with no supervised completion tokens.

For weighted relevance and functional training, the scripts construct a vocabulary-level weight vector. A class with count n_c among N examples receives $w_c = N/(Kn_c)$ at the selected label-token identifier, while other tokens retain weight one. The single-token variant maps relevance labels to four letters and asserts that each maps to one token. The functional variant weights the first token of each verbal label. This is token-weighted cross-entropy, not a clean example-level loss weighting when labels have multiple tokens or share a token. We preserve that implementation distinction in interpreting the trials.

The extraction batch is $4 \times 4 = 16$, despite a stale source comment claiming 32. Its script enables gradient checkpointing. The dedicated functional and letter-label runs use explicit warmup-step counts; all listed scripts use cosine learning-rate schedules. Available v1 trainer state ends at epoch 0.454, extraction at 2.041, functional at approximately 66.71, and single-token relevance at 0.48. Update counts therefore correspond to very different data exposure. There are no repeated-seed estimates or matched compute budgets across configurations.

4.3 Checkpoint evaluation is monitoring, not adaptive training

Callbacks evaluate development samples periodically: preserved v1 monitoring uses small per-task samples; current shared code caps each task at 100, single-token relevance uses 150 rows, and extraction uses 100. Functional monitoring uses all 28 development rows. These samples differ from full development evaluations. Callbacks record scores and compare them with stored comparator numbers, but do not implement a validated automatic curriculum, loss controller, rollback policy, or adaptive stopping rule. Some stored comparator numbers come from different samples, so their comparison flags cannot be used as release gates. Section 5 formalizes the missing conditions.

5 Evaluation Contracts and Audit Procedure

We represent an evaluation by

$$\mathcal{E} = (D, I, X, Y, P, G, S, H), \quad (3)$$

where D identifies the data version and partition, I the ordered example identifiers, X the evidence exposed to the model, Y the label ontology, P the prompt/decoding protocol, G the gold labels, S the scoring program, and H the history of training and selection exposure. This is a reporting

contract, not a new learning algorithm. In particular, equal metric names or equal sample sizes do not imply equal evaluations.

A paired model comparison requires aligned (D, I, X, Y, G, S) and documented, task-appropriate choices of P for each model. It need not force identical syntax on models trained with different interfaces, but any prompt adaptation must preserve the underlying evidence and be selected without using test outcomes. H distinguishes a diagnostic sample from an untouched generalization test. For a leaderboard claim, the contract must additionally match the publisher’s official split and metric. For a cost comparison, it must include request lengths, decoding budgets, hardware, concurrency, warmup, and the accounting interval.

The offline audit performs four operations. It checks file hashes against the original inventory; reconstructs counts and overlap keys from every row in the analyzed data files; recomputes metrics where predictions, confusion matrices, or sufficient counts exist; and labels missing information explicitly. It verifies 118 source inputs, including preserved local artifacts. Reproducibility here means that another reader can regenerate the reported analysis from the released evidence. It does not mean that every historical model run can be replayed bit-for-bit.

5.1 Scoring semantics and counterexamples

The shared-adaptor evaluation marks an answer correct when the lowercased gold string is a substring of the generated answer. Formally,

$$S_{\text{sub}}(y, \hat{y}) = \mathbb{K}[\text{lower}(y) \subseteq \text{lower}(\hat{y})]. \quad (4)$$

This rule accepts gold *compatible* with prediction *incompatible*. It also accepts gold *same* in *not same*. These are executable counterexamples to the scorer, not reconstructed predictions from missing model outputs. In particular, the saved 1.0 compatibility score cannot certify perfect label accuracy. The raw outputs needed to rescore that historical result are absent.

Extraction scoring accepts a nonempty field when either the predicted string contains the gold string or vice versa. It computes field-level precision, recall, and F1 from the resulting counts. Thus *black* can match *black suede*, and an overlong answer containing the gold value can receive credit. We retain the original metric for traceability and call it *permissive field F1*. It should be supplemented by normalized exact match and evidence-grounding evaluation in a future run.

The functional eCeLLM evaluator scans for the first character in a, b, or c, mapping that character to a relation. Its 28 saved outputs consist of ten instances of a: b and eighteen malformed JSON-like fragments. A character inside an unrelated word can therefore become a label. The stored parser reports four correct labels and no invalid outputs; a strict full-answer check accepting one letter with optional trailing punctuation accepts 0/28. This demonstrates format noncompliance under this particular evaluation setup. It does not establish that the model intrinsically cannot solve functional relation.

5.2 API evidence and benchmark scope

The saved API relevance/identity report includes per-example parsed labels, allowing its accuracy to be recomputed. A preflight records resolved provider model names, but the scoring calls use configured model strings and do not retain the returned model identifier for each response. We therefore use the historical aliases in the tables without certifying a frozen provider snapshot. Output-budget policies also differ between providers. Appendix C details these limitations.

Public datasets are used as sources rather than as completed official benchmark submissions. No query-level ranking metric, full ESCI leaderboard score, complete WDC generalization grid, or general retail task suite is available. Compatibility and functional relation are small project-specific diagnostics, not public benchmarks. We make no assertion that these results beat all public, open-weight, or contemporary frontier systems.

Table 5: Logged shared-adapter full-development scores under the historical substring scorer. Parentheses give sample size. Rows do not share one complete evaluation contract. Compatibility values are particularly compromised by the label-containment bug.

Run	Relevance	Identity	Functional	Compatibility
v1 rescore	0.527 (1,000)	0.916 (3,500)	0.692 (13)	1.000 (8)
v2	0.491 (1,000)	0.914 (3,500)	1.000 (15)	1.000 (4)
v3	0.503 (4,000)	0.888 (3,500)	0.133 (15)	1.000 (4)

Table 6: API and eCeLLM measurements on the separate balanced relevance/identity samples. API counts are recomputed from saved parsed labels; eCeLLM counts are reconstructed from aggregates. The shared adapter has no saved evaluation on these same 60/50 rows, so no adapter-versus-API difference is calculated.

Model label	Relevance correct	Accuracy	Identity correct	Accuracy
claude-haiku-4-5	28/60	0.467	39/50	0.780
claude-sonnet-5	35/60	0.583	46/50	0.920
gpt-5-mini	29/60	0.483	40/50	0.800
gpt-5	28/60	0.467	42/50	0.840
eCeLLM-S (aggregate)	17/60	0.283	31/50	0.620

6 Results and Error Analysis

6.1 RQ1: shared adaptation and task-specific behavior

Table 5 reports the complete logged full-development shared-adapter results. The v1 rescore reaches 0.527 relevance and 0.916 identity under the historical substring scorer. The prior manuscript reported 0.526 and 0.914; its original complete evaluation vector is unavailable, so those values are retained in the archive rather than silently equated with the later rescore. Neither score should be substituted for a matched result on the API sample.

The shared experiments change several factors at once: relevance sampling, mixture size, update budget, rank, and in some cases evaluation population. Their order documents the development process, but the observed differences cannot isolate the causal effect of any one change. In particular, the v3 functional decrease occurs on a 15-row single-class diagnostic; it motivates investigation of task interference but does not by itself demonstrate catastrophic forgetting of a broad capability. No pre/post general-ability suite or controlled gradient-conflict experiment is available.

The API scores in Table 6 are independently useful reference measurements on those samples. They cannot support claims that v1 beats three APIs, ties the strongest identity model, or falls short by an exact number of points. The adapter’s identity majority baseline of 0.857 also differs from the balanced API sample’s baseline of 0.5. Subtracting accuracies across those populations would conflate model quality with class prevalence and difficulty.

6.2 Single-token relevance: avoiding one collapse is not balanced performance

The dedicated letter-label adapter’s full 4,000-row confusion matrix supports a deeper analysis than aggregate accuracy alone. Accuracy is 0.5115, macro-F1 is 0.436, and balanced accuracy is 0.414. All four classes are predicted, but complement recall is only 0.200 and irrelevant recall 0.304 (Table 7). Substitute predictions occupy 1,939/4,000 outputs although only 1,381 gold rows have that class.

Of 175 true complements, 105 are predicted substitute; of 691 irrelevant pairs, 362 receive that prediction. The pattern is compatible with an overly broad substitute decision region, but no representation-level analysis was performed. Earlier free-form specialist attempts are described in development notes as class collapse on small inspected samples; their complete prediction vectors are absent. The available evidence therefore establishes the letter model’s final error structure, not a controlled proof that token length caused the earlier failures.

Table 7: Recomputed single-token relevance metrics. Support and predicted counts expose errors hidden by the headline accuracy. These are development-set results, not an untouched test estimate.

Class	Support	Predicted	Precision	Recall	F1
exact	1753	1608	0.616	0.565	0.589
substitute	1381	1939	0.418	0.587	0.489
complement	175	86	0.407	0.200	0.268
irrelevant	691	367	0.572	0.304	0.397
Macro average				0.414	0.436

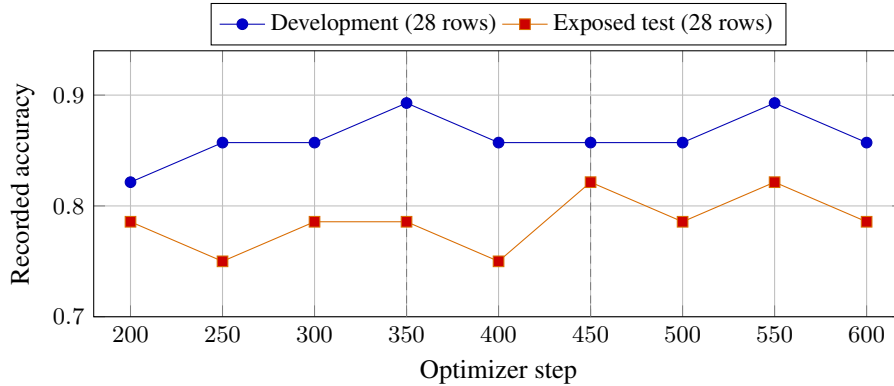


Figure 2: Dedicated functional adapter: all nine checkpoints in the saved test sweep. Step 350 is the earliest development maximizer; step 450 is the published checkpoint selected after test inspection. The difference in their test score is one example. Scenario overlap and repeated test access prevent interpreting this curve as an untouched generalization evaluation.

6.3 Functional relation: exposure and selection dominate the interpretation

The dedicated functional adapter’s published checkpoint reaches $23/28 = 0.821$ on the partition named `locked_test`, with macro-F1 0.780. Its confusion matrix shows all 14 complements correct, seven of ten substitutes correct, and two of four unrelated examples correct; every remaining prediction is complement. The test partition is small and predominantly shares training scenarios.

Checkpoint selection creates a second limitation. Development accuracy is maximal at steps 350 and 550 (25/28). A retrospective earliest-tie development rule would select step 350, which obtains 22/28 on the exposed test. The saved sweep evaluates nine checkpoints on that test, and the published step 450 obtains 23/28. The one-example increase is thus part of a test-informed selection process. It is not an unbiased final evaluation of a rule fixed beforehand. Figure 2 retains every saved test-sweep value.

All four APIs obtain 11/13 on the earlier functional diagnostic, but that is a different set from the dedicated adapter’s 28 rows. In addition, the API prompt constructor takes the first two newline-separated strings of the generated description; two of the 13 second-listing fields are empty because of blank lines. Consequently, neither the apparent development advantage nor the apparent test deficit relative to 0.846 is a defensible paired model difference. The correct next comparison requires new scenario-disjoint examples and identical complete evidence.

6.4 RQ2: extraction provides the clearest matched signal

Table 8 compares systems on the same 200-listing extraction sample with the recorded permissive field scorer. This is the strongest common-population comparison in the artifact. The adapter ties GPT-5-mini and GPT-5 at brand F1 1.000, and its color F1 0.955 is 1.38 points above the best API color result, Haiku at 0.941. The earlier narrative incorrectly identified GPT-5’s 0.931 as the strongest color baseline. Counts, not rounded differences, determine the corrected ordering.

Table 8: Extraction on the same 200 rows. Counts are TP/FP/FN under permissive field matching, not strict string equality. All F1 values are recomputed from counts. Two evaluation texts overlap training; per-example prediction vectors are unavailable for decontamination and paired F1 inference.

System	Brand counts	Brand F1	Color counts	Color F1
Dictionary	176/24/24	0.880	133/20/67	0.754
GLiNER2	153/41/47	0.777	167/24/33	0.854
GLiNER2.5	148/45/52	0.753	166/15/34	0.871
claude-haiku-4-5	193/7/7	0.965	184/7/16	0.941
claude-sonnet-5	199/0/1	0.997	180/9/20	0.925
gpt-5-mini	200/0/0	1.000	182/11/18	0.926
gpt-5	200/0/0	1.000	183/10/17	0.931
Dedicated adapter	200/0/0	1.000	191/9/9	0.955

For each field, $F_1 = 2TP/(2TP + FP + FN)$. A missing prediction contributes a false negative; an incorrect nonempty value can contribute both a false positive and a false negative. The adapter records zero JSON parse failures on this sample. Brand is an easier field here in the sense that every gold value occurs literally in the supplied text; this observation is specific to the selected data, not a general claim about brand extraction difficulty.

The result supports a promising specialization signal but not a statistically established advantage. The sample contains only 200 listings, two exact texts overlap training, and the report retains aggregate counts without the raw paired field outputs. We cannot determine the corrected score after removing overlaps, stratify errors by evidence availability, or bootstrap paired F1 without inventing missing outcomes. It is also not possible to infer that a stronger baseline tuned on the same training data would retain the same gap: the GLiNER variants were used as off-the-shelf baselines.

6.5 What aggregate counts imply about paired uncertainty

A useful diagnostic can still be derived without reconstructing nonexistent predictions. Treat a color true positive as binary correctness on each of the $n = 200$ rows. The adapter has $a = 191$ correct and Haiku has $b = 184$. Let x be the unobserved number both get correct. The feasible overlap is

$$\max(0, a + b - n) \leq x \leq \min(a, b), \quad 175 \leq x \leq 184. \quad (5)$$

The discordant counts are $(a - x, b - x)$. Under the usual paired binary-test assumptions, the two-sided exact McNemar probability for any feasible pairing is

$$p(x) = \min \left\{ 1, 2 \sum_{k=0}^{\min(a-x, b-x)} \binom{a+b-2x}{k} 2^{-(a+b-2x)} \right\}. \quad (6)$$

Enumerating all feasible integer x yields $p \in [0.0156, 0.2295]$. Some possible pairings cross a conventional 0.05 threshold and others do not. Thus the marginal counts alone cannot determine even this binary paired-test conclusion. This range is *not* an observed p-value, a confidence interval, or a paired F1 test. It also does not repair training overlap or dependence among related listings. The calculation illustrates why preserving raw paired predictions is scientifically necessary even when point estimates look favorable.

6.6 Open-weight comparator coverage

The artifact contains useful comparator attempts but not an exhaustive matched Hugging Face benchmark. eCeLLM relevance/identity reports contain aggregates; its functional raw-output failure was analyzed in Section 5.1. The Qwen embedding and Rex reranker sweep fits identity thresholds on the evaluation sample itself. Its 0.70/0.72 identity accuracies are optimistic selection diagnostics over the tried thresholds, not independently calibrated test scores. Its relevance numbers evaluate a binary relevant/irrelevant reduction, whose positive-class majority baseline is 0.75; they cannot be compared with four-class relevance accuracy.

RexBERT without a fitted classification head is not a completed supervised comparator. NuExtract’s missing completed evaluation is an execution gap, not proof that text-only extraction is inapplicable.

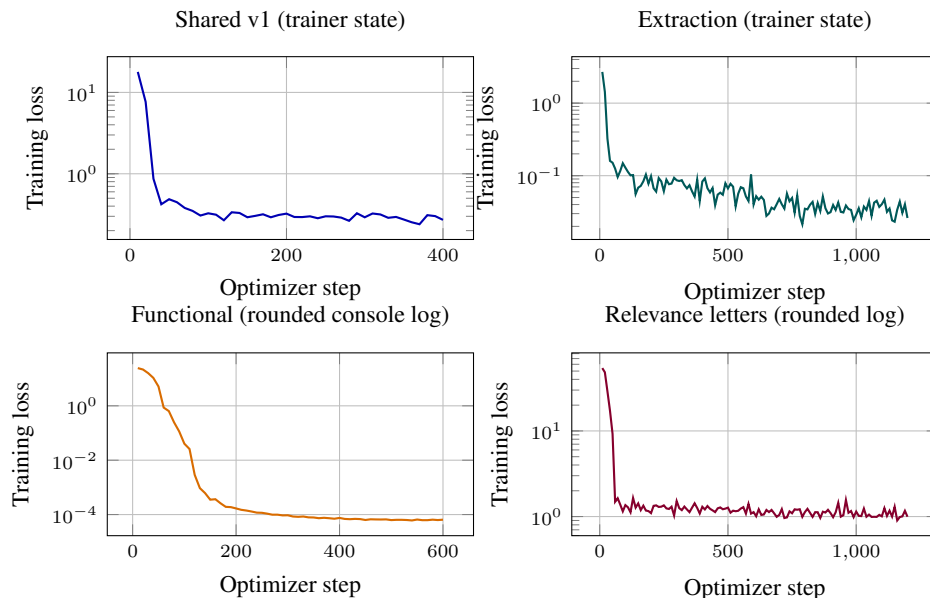


Figure 3: All saved ten-step training-loss records for the four runs with available loss traces. Each panel has its own logarithmic vertical scale. Loss magnitudes differ across objectives and must not be ranked as task quality. Decreasing functional loss is compatible with limited generalization and repeated exposure to 131 training rows.

Comparator registries and status manifests are partly stale and do not substitute for prediction artifacts. We consequently retain a distinction between *listed*, *attempted*, *measured*, and *comparably evaluated* systems throughout the paper.

7 Training Dynamics and Serving Evidence

7.1 RQ3: optimization progress versus useful quality

Figure 3 shows all available loss records for four runs. Shared v1 and extraction use trainer-state records; functional and single-token relevance use the rounded values printed in console logs at ten-step intervals. The functional loss approaches zero while test behavior remains imperfect and scenario-dependent. Extraction loss also falls rapidly, whereas its monitored color accuracy varies between checkpoints. These observations motivate evaluating task behavior throughout training rather than using decreasing loss as a proxy for generalization.

Loss magnitudes are not directly comparable across panels: objectives differ in token weighting, completion length, and data distribution. The functional log reports approximately 389.2 seconds of trainer runtime and the single-token relevance log approximately 1,004 seconds. These are measurements from individual runs, not end-to-end campaign cost, and do not include every failed experiment, provisioning interval, upload, API call, or idle period. Exact GPU billing and a complete per-run environment ledger are absent. We therefore do not derive a dollars-per-quality or hardware Pareto frontier from these durations.

7.2 Implemented service and observed load-test limits

The FastAPI implementation has relevance, identity, functional-relation, and catalog-normalization routes. It lazily loads three separate base-plus-adapter model instances for shared Match, functional relation, and Understand. The process has per-loader locks intended to prevent duplicate concurrent loading. This is different from keeping one resident base and switching adapters, and no memory comparison between the two strategies was measured. A technical-compatibility inference route is not present in the inspected API.

Table 9: Completed requests in retained load-test statistics. No model-latency percentile or inference throughput is estimable from these completions. Zero logged failures does not imply that pending requests succeeded.

Artifact	Health completed	Inference completed	Logged failures	Inference p50/p99
Before lock	1	0	0	Not observed
After lock	0	0	0	Not observed

The retained Locust CSVs are decisive about what was measured (Table 9). Before the loading-lock change, the statistics contain one completed health request and no completed model inference. The after-change CSV contains zero completed requests. The accompanying narrative describes a three-user, 90-second CPU test and an isolated slow functional response, but the raw server traces needed to reproduce the loading diagnosis are not archived. The code contains the lock; the CSVs do not demonstrate recovered inference throughput.

A health response checks adapter-path availability, not a completed warm inference or a readiness guarantee. Requests that remain unfinished at the end of a test are censored observations; assigning them zero latency, zero error probability, or the health route’s latency would be incorrect. No GPU concurrency curve, sustained request-rate measurement, completed-request tail latency, or production availability estimate is established by these artifacts.

7.3 Cost accounting must include successful task completion

The service exposes an estimate based on a configured \$0.74/hour rate and an assumed 20 tokens/second. Its resulting \$0.0103 per 1,000 tokens is arithmetic over assumptions, not an observed serving cost. Training iterations per second do not validate decoding tokens per second, and prompt processing and generation cannot generally be priced by a single assumed throughput. We exclude this estimate from comparative results.

For a future serving evaluation, a useful task-level quantity is

$$C_{\text{valid,SLO}} = \frac{C_{\text{GPU}} + C_{\text{CPU}} + C_{\text{storage}} + C_{\text{network}} + C_{\text{API}}}{\sum_{i=1}^N \mathbb{1}[\text{request } i \text{ correct, schema-valid, and within SLO}]}, \quad (7)$$

where all billed resources, including loading and idle capacity, are counted over the same declared interval. A zero denominator is undefined rather than a favorable estimate. Reporting this alongside raw cost/request, completion rate, p50/p95/p99 latency, and throughput separates low unit price from useful service. Equation 7 is a proposed accounting criterion; this study does not supply the measurements needed to instantiate it.

8 Discussion, Limitations, and Responsible Use

What the evidence supports. Small-model specialization appears promising for this narrow extraction sample: it produces valid JSON and favorable field F1 relative to the measured off-the-shelf and API baselines. The matching experiments demonstrate that one shared adapter can emit several task labels, but they do not establish a uniformly strong horizontal commerce model. The letter-label relevance analysis illustrates why measuring all classes is necessary even after obvious output collapse disappears. The functional analysis shows that good development accuracy can coexist with scenario reuse and test-informed selection.

What makes the analysis useful beyond this artifact. Four failures arise at different levels: shared wording creates scorer collisions; paraphrases share scenario-level information; post-fit repartitioning fails to remove prior exposure; and aggregate score tables lose the pairing required for inference. Their remedies require different units of control. A strict parser cannot fix scenario overlap, and more data cannot repair an exposed test set. The comparison contract makes these dependencies explicit before a result is promoted into a model-selection or deployment claim. The aggregate-pairing calculation is a compact diagnostic of an additional information loss: even correct marginal counts can leave a significance conclusion unidentified.

Limits to generalization and novelty. The study has one base size, no repeated seeds, limited category coverage, and no measured multilingual, merchant-held-out, temporal, or adversarial robustness. Unknown/abstention behavior and calibration are not evaluated. The underlying public corpora may have appeared during pretraining; that exposure is unmeasured. The strongest comparison uses a permissive scorer and a small partially overlapping sample. The synthetic judge is neither blinded nor human, and no naturalness annotation is available. No claim of being the first commerce SLM, of beating all related Hugging Face models, or of inventing multitask QLoRA follows from this evidence. Novel training, data-selection, and serving mechanisms would require implementation and controlled evaluation beyond the present record.

Experiments needed to strengthen the result. The highest-priority extension is a new, frozen evaluation suite with query-, entity-, and scenario-level separation as appropriate; strict label parsers; exact and normalized extraction metrics; and retained per-example outputs. Shared and specialist models should then be trained with matched data, budgets, and multiple seeds, alongside an unfine-tuned Qwen baseline and supervised compact alternatives. A second phase should vary real/synthetic mixture, scenario grouping, label representation, and task balancing one factor at a time. Finally, GPU serving measurements should evaluate quality-preserving precision and batching choices under declared service-level objectives. These extensions are prospective, not results hidden behind the current tables. Appendix F specifies the comparisons in more detail.

Responsible use and data rights. Product matching errors can merge distinct offers or misrepresent fit; functional complementarity must not be treated as a guarantee of technical compatibility. The present diagnostics do not justify autonomous compatibility assurance. The input corpora carry their respective publisher terms, and a repository manifest is not a substitute for reviewing redistribution and commercial-use conditions. The experiments use product data and synthetic descriptions rather than customer conversations; that does not establish a comprehensive privacy audit. No human annotation study was conducted in this work. Deployment would require task-specific validation, evidence-aware handling of uncertain outputs, and appropriate operational controls; these capabilities are not established by a request schema containing a tenant identifier.

Research assistance disclosure. Generative AI assisted the manuscript revision, code inspection, literature synthesis, and implementation of the offline evidence audit. Synthetic training-data generation and automated judging are described separately in Section 3. The new analyses use existing artifacts rather than fabricated model outputs. Responsibility for validating the manuscript and any eventual submission remains with the author.

9 Conclusion

CommerceCore Expansion provides a small-model adaptation case study whose strongest supported signal is brand/color extraction on a common 200-listing sample. Reconstructing the full evidence changes the interpretation of the broader campaign: matching comparisons are constrained by unequal samples, minority diagnostics reuse scenarios, checkpoint selection exposes a nominal test partition, and serving traces contain no completed inference load measurements. The released audit makes these boundaries reproducible and adds class-level analysis, scorer checks, and aggregate paired-uncertainty bounds. A credible path toward a broadly useful commerce model requires preserving evaluation independence and operational measurements as carefully as the model weights themselves.

References

- [1] Arghya Mukherjee. CommerceCore: prior query-parsing model and research artifacts. Public project manuscript, 2026. <https://github.com/arghya05/commercecore/tree/main/paper>.
- [2] Bo Peng, Xinyi Ling, Ziru Chen, Huan Sun, and Xia Ning. eCeLLM: Generalizing Large Language Models for E-commerce from Large-scale, High-quality Instruction Data. In *ICML*, 2024. <https://arxiv.org/abs/2402.08831>.

- [3] Yangning Li, Shirong Ma, Xiaobin Wang, Shen Huang, Chengyue Jiang, Hai-Tao Zheng, Pengjun Xie, Fei Huang, and Yong Jiang. EcomGPT: Instruction-tuning Large Language Models with Chain-of-Task Tasks for E-commerce. 2023. <https://arxiv.org/abs/2308.06966>.
- [4] Xinyi Ling, Hanwen Du, Bo Peng, Zhihui Zhu, and Xia Ning. Captions Speak Louder than Images: Generalizing Foundation Models for E-commerce from High-quality Multimodal Instruction Data. In *IJCNLP-AACL*, 2025. <https://arxiv.org/abs/2410.17337>.
- [5] Josip Tomo Licardo and Nikola Tanković. Performance Trade-offs of Optimizing Small Language Models for E-Commerce. 2025. <https://arxiv.org/abs/2510.21970>.
- [6] Chandan K. Reddy, Lluís Màrquez, Fran Valero, Nikhil Rao, Hugo Zaragoza, Sambaran Bandyopadhyay, Arnab Biswas, Anlu Xing, and Karthik Subbian. Shopping Queries Dataset: A Large-Scale ESCI Benchmark for Improving Product Search. 2022. <https://arxiv.org/abs/2206.06588>.
- [7] Ralph Peeters, Reng Chiz Der, and Christian Bizer. WDC Products: A Multi-Dimensional Entity Matching Benchmark. In *EDBT*, 2024. <https://arxiv.org/abs/2301.09521>.
- [8] Yuliang Li, Jinfeng Li, Yoshihiko Suhara, AnHai Doan, and Wang-Chiew Tan. Deep Entity Matching with Pre-Trained Language Models. *Proceedings of the VLDB Endowment*, 14(1):50–60, 2020. <https://arxiv.org/abs/2004.00584>.
- [9] Urchade Zaratiana, Gil Pasternak, Oliver Boyd, George Hurn-Maloney, and Ash Lewis. GLiNER2: An Efficient Multi-Task Information Extraction System with Schema-Driven Interface. 2025. <https://arxiv.org/abs/2507.18546>.
- [10] Jasmine Collins, Shubham Goel, Kenan Deng, Achleshwar Luthra, Leon Xu, Erhan Gundogdu, Xi Zhang, Tomas F. Yago Vicente, Thomas Dideriksen, Himanshu Arora, Matthieu Guillaumin, and Jitendra Malik. ABO: Dataset and Benchmarks for Real-World 3D Object Understanding. In *CVPR*, 2022. <https://arxiv.org/abs/2110.06199>.
- [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-Rank Adaptation of Large Language Models. In *ICLR*, 2022. <https://arxiv.org/abs/2106.09685>.
- [12] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient Finetuning of Quantized LLMs. In *NeurIPS*, 2023. <https://arxiv.org/abs/2305.14314>.
- [13] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks. In *ICML*, 2018. <https://arxiv.org/abs/1711.02257>.
- [14] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient Surgery for Multi-Task Learning. In *NeurIPS*, 2020. <https://arxiv.org/abs/2001.06782>.
- [15] Sayash Kapoor and Arvind Narayanan. Leakage and the Reproducibility Crisis in ML-based Science. 2022. <https://arxiv.org/abs/2207.07048>.
- [16] Marco Tulio Ribeiro, Tongshuang Wu, Carlos Guestrin, and Sameer Singh. Beyond Accuracy: Behavioral Testing of NLP Models with CheckList. In *ACL*, 2020. <https://arxiv.org/abs/2005.04118>.

A Artifact Provenance and Reproduction

A.1 What is preserved

The evidence inventory records the 106 originally tracked non-paper files at baseline commit `c7deee76caa8c22058088d501befc32c57513d41`. The audit reads and verifies the original run-time, dataset, report, and manifest files against those hashes; the repository README may be updated to reflect this manuscript. Additional archived evidence includes the original shared development rows, v1 and extraction trainer states, the functional checkpoint history, four adapter configurations, and the previous manuscript, evidence builder, and macros. The archive manifest distinguishes original files from analysis generated in this revision.

The audit’s output records hashes for each source it reads. Numerical TeX rows and plot CSVs are generated from these inputs, while methodological prose is supported by source inspection. The paper source package includes every TeX/style/CSV dependency required to compile the manuscript; the full repository supplies the underlying data and reports needed to regenerate the analysis. Model weights are not required to execute the offline audit.

Table 10: Evidence map. Paths are relative to the repository; abbreviated report directories are dated in the committed inventory.

Claim or analysis	Primary evidence
Mixture and split counts	<code>data/training_mixture_final.jsonl</code> ; <code>data/training_mixture_v3.jsonl</code>
Historical v1 split	<code>paper/evidence/archive/local_artifacts/models/shared_adapter_v1/dev_rows_for_eval.json</code>
Scenario reuse	<code>data/synthetic_match/functional_relation_v3_final.jsonl</code>
Extraction overlap	<code>data/understand_train.jsonl</code> ; <code>data/understand_dev.jsonl</code> ; <code>data/abo/understand_eval_grounded.json</code>
API parsed predictions	<code>reports/frontier_comparison_2026-09-27/report.json</code>
Extraction aggregate counts	<code>reports/understand_locked_eval_result.json</code> ; <code>reports/understand_frontier_comparison_2026-09-28/report.json</code>
Functional checkpoint selection	<code>reports/functional_relation_locked_eval_checkpoint_sweep.json</code>
Single-token confusion	<code>reports/relevance_singletoken_experiment_2026-09-28/final_full_dev_scores_singletoken.json</code>
Serving completions	<code>reports/load_test_2026-09-28/result_stats.csv</code> ; <code>result_after_fix_stats.csv</code> in the same directory

A.2 Reproduction commands and boundaries

From the repository root, `python3 paper/audit_evidence.py` regenerates the audit JSON, metric macros, table rows, and plot CSVs using only the Python standard library. The unit checks in `paper/test_audit_evidence.py` exercise the metric reconstruction, scorer counterexamples, and aggregate pairing enumeration. `python3 paper/build_paper.py` runs the audit, compiles the manuscript with Tectonic, checks the log, and packages the sources. A fresh compilation of the extracted source ZIP verifies that the paper does not depend on unbundled local figure files.

This workflow reproduces the analysis, not the unavailable raw generations or historical software environments. Trainer scripts contain version-sensitive arguments and were edited between runs; an early smoke log records an unsupported `warmup_ratio` argument, followed by scripts using explicit warmup steps. A source comment records an extraction batch reduction after an out-of-memory event, but a complete OOM trace is not archived. Dependency ranges, incomplete hardware meta-data, and missing model-load revision enforcement prevent a stronger exact-replay claim. These constraints are disclosed rather than filled with estimated settings.

Table 11: Selected split checks. A zero in one key space is not proof of independence in another. Scenario equality uses a hash of canonicalized structured metadata; text checks use exact or explicitly normalized text as described in the audit.

Check	Shared keys / affected rows	Evaluation rows
Current relevance train/dev query IDs	0 shared keys	1,000
Scaled relevance train/dev query IDs	0 shared keys	4,000
Expanded functional dev: training scenario seen	26 affected rows	28
Expanded functional test: training scenario seen	25 affected rows	28
Corrected functional dev: earlier train text seen	11 affected rows	13
Corrected compatibility dev: earlier train text seen	6 affected rows	8
Extraction train/dev item IDs	4 shared keys	300
Extraction train/dev exact text	2 shared keys	300
Extraction train/eval item IDs	0 shared keys	200
Extraction train/eval exact text	2 shared keys	200

B Additional Data and Comparison Diagnostics

The current 8,000-row relevance training component has 3,405 exact, 2,849 substitute, 382 complement, and 1,364 irrelevant labels. Its 1,000-row development component has 435, 303, 60, and 202 respectively. The scaled 40,000-row training set contains 17,401 exact, 14,071 substitute, 1,850 complement, and 6,678 irrelevant labels; the corresponding 4,000-row development counts are 1,753, 1,381, 175, and 691. This distribution shift alone makes raw differences across the two development populations difficult to attribute to a model change.

The expanded functional training labels are 63 complement, 47 substitute, and 21 unrelated. Both 28-row evaluation partitions contain 14 complement, ten substitute, and four unrelated. A majority-complement classifier obtains 0.5 on each. Although the dedicated adapter exceeds that reference in the recorded results, scenario overlap prevents treating all 28 rows as independent novel scenarios. The exact number of statistically independent examples cannot be inferred from row count alone.

B.1 Comparison status ledger

Table 12: Interpretation of prominent historical claims after applying the evaluation contract.

Comparison	Available evidence	Defensible interpretation
Adapter vs APIs, relevance/identity	Different sample IDs and label prevalence	Separate descriptive scores
Adapter vs APIs, extraction	Common sample and permissive field metric	Favorable point estimate; no paired inference
Shared vs dedicated functional	Different splits and scenario exposure	Separate diagnostics; no isolated adaptation gain
Compatibility perfection	Gold substring scorer accepts opposite label	Strict accuracy unknown
eCeLLM functional loss	All raw answers fail a strict letter parser	Evaluation-format failure, not fair capability ranking
Scaled mixture causes forgetting	Multiple changes; single-class monitoring	Interference hypothesis, not causal demonstration
Serving cost/latency advantage	No completed inference in load CSVs	Advantage unmeasured

C Prompt, Decoding, and Scoring Details

C.1 Training and model prompts

The shared adapter receives a plain-text instruction followed by the stored input and `Answer:`. For relevance, the instruction is “Classify the query-product relevance: exact, substitute, complement, or irrelevant.” Identity asks whether the listings describe the same purchasable item or are distinct.

Shared functional and compatibility prompts additionally mention *unknown*, despite that class being absent from their training labels. This discrepancy is preserved in the audit and should be resolved in a future ontology-controlled run.

Extraction uses the instruction to extract brand and color and respond with only a JSON object. The API extraction prompt explicitly permits null when a field is not mentioned or unclear; the adapter prompt does not include that additional sentence. Both see the listing text, but the null instruction is a prompt-policy difference that should be aligned or ablated before asserting an intrinsic model advantage. Raw JSON parse outcomes are counted as missing field predictions by the retained scorer.

The API relevance prompt supplies natural-language definitions of all four classes, unlike the shorter adapter instruction. The identity prompt emphasizes the purchasable variant. These are reasonable model-interface choices, but they are not identical prompts or evidence of a controlled prompting study. The source scripts contain the complete templates; no prompt search history or equal tuning budget is archived.

Table 13: Preserved decoding settings. Unspecified values follow the installed model/provider defaults and cannot be reconstructed as fixed cross-provider settings.

Path	Generation settings	Important missing control
Shared development	Greedy; up to 6 new tokens	Raw full-dev generations
Extraction adapter	Greedy; up to 40 new tokens	Raw per-row JSON and score vector
Anthropic Match calls	Up to 10 output tokens; 30-second timeout	Matched reasoning/output budget
Anthropic extraction	Up to 100 output tokens; 30-second timeout	Per-response usage and revision record
OpenAI comparator calls	30-second timeout; no explicit output cap in inspected calls	Explicit budget and returned revision per response

The preflight resolution report lists Haiku as `claude-haiku-4-5-20251001`, Sonnet as `claude-sonnet-5`, GPT-5-mini as `gpt-5-mini-2025-08-07`, and GPT-5 as `gpt-5-2025-08-07`. These are recorded experiment metadata, not a claim that they are the latest available models. The scoring functions use their configured model strings; in particular, the OpenAI calls use aliases rather than the preflight’s dated strings. Parsed predictions and aggregate reports alone cannot prove which backend served every call.

C.2 Scorer stress cases

A small deterministic test suite can establish that a scorer is unsound without assuming that a particular model emitted the offending answer. The cases in Table 14 are constructed fixtures. They do not count as additional benchmark examples or model failures. The audit tests execute the retained extraction scoring function without importing its GPU dependencies and isolate the shared evaluator’s actual correctness expression for the classification fixtures.

Table 14: Constructed scorer counterexamples and one retained raw-output pattern.

Gold	Answer	Historical treatment	Required distinction
compatible	incompatible	Correct substring	Opposite labels
same	not same	Correct substring	Negated assertion
black suede	black	Correct extraction field	Partial versus exact value
Any letter label	a: b	First letter becomes label	Ambiguous/malformed answer

Table 15: Single-token relevance development confusion matrix. Rows are gold labels and columns predictions; total 4,000.

Gold / predicted	exact	substitute	complement	irrelevant
exact	990	661	18	84
substitute	500	811	8	62
complement	24	105	35	11
irrelevant	94	362	25	210

Table 16: Published functional checkpoint confusion matrix on the exposed 28-row test partition.

Gold / predicted	substitute	complement	unrelated
substitute	7	3	0
complement	0	14	0
unrelated	0	2	2

D Complete Available Error and Checkpoint Analyses

D.1 Confusion matrices

D.2 Monitoring trajectories

Figure 4 preserves the recorded monitoring histories separately from the full-development results. V1/v2 relevance and identity monitoring uses 30-row samples, while the v3 histories use larger samples consistent with the later 100-row cap. Minority-task monitoring uses the available 15/4-row historical development partitions. These differences prohibit treating the curves as a matched data-scaling ablation. They still show why a single final score is an incomplete account of training behavior.

The letter-label callback’s recorded values differ substantially from its final full-development score. It uses a 150-row sample and the inherited substring-based callback rather than the final parsed-label confusion evaluation. With no paired callback generation vector retained, we do not attribute that discrepancy solely to sampling, parsing, or model state. It is a further reason to preserve one versioned evaluator and raw predictions throughout a controlled run.

The extraction monitor uses 100 sampled development rows and field-correctness rates, not the 200-row final field-F1 evaluation. Its color monitor reaches 0.94 at step 900 and 0.93 at step 1,200, while its brand monitor remains 1.0 throughout. The saved final model is the end-of-run adapter, so a best-development checkpoint policy should not be retroactively asserted.

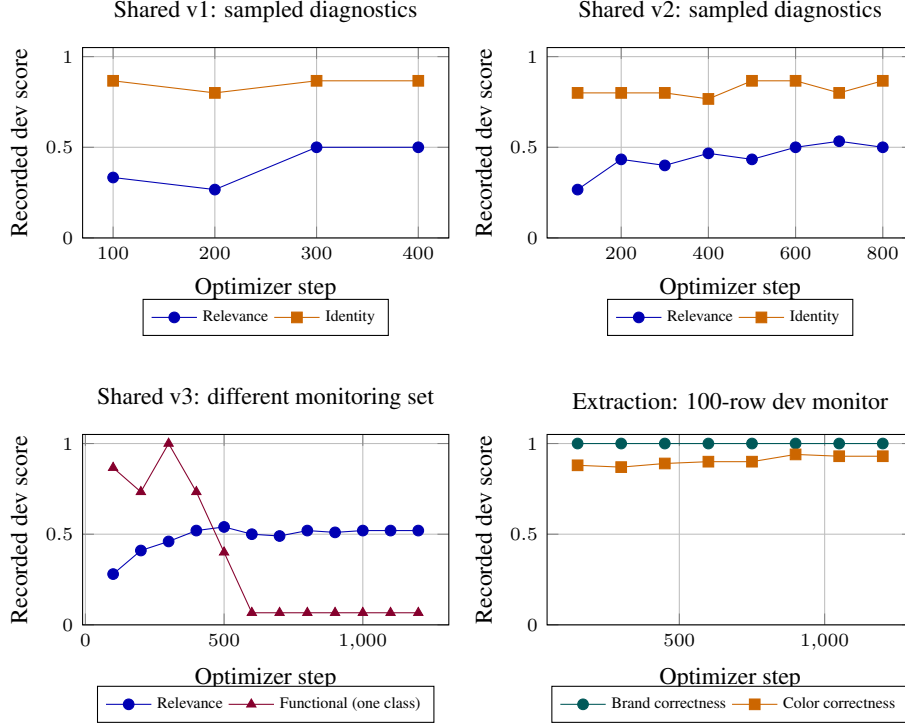


Figure 4: Monitoring trajectories from saved histories. Each panel represents its own evaluation sample and scorer; no cross-model API threshold is overlaid. The v3 functional monitor and the final batched score differ (0.067 versus 0.133), underscoring that recorded evaluations cannot be silently merged. Full machine-readable histories, including single-token and compatibility values, accompany the paper.

Table 17: Loss-trace endpoints. Functional console loss is rounded in the table; the underlying final printed value is approximately $6.477 \cdot 10^{-5}$. Traces are not matched objectives.

Run	Records	First logged loss	Last logged loss	Last logged epoch
shared v1	40	17.8771	0.2712	0.454
understand	120	2.6950	0.0261	2.041
functional	60	24.4200	0.0001	66.710
single token	120	54.1600	0.9950	0.480

D.3 Interpretation of earlier specialist attempts

The repository narrative records an unweighted dedicated relevance trial and a weighted verbal-label trial, including small manually inspected samples dominated by *exact*. These observations motivated the letter-label run. Their full per-example outputs, stable environment snapshots, and a matched final evaluation are not retained. The manuscript therefore reports them as exploratory development observations, not as five independently falsified scientific hypotheses. The preserved code also shows that the latest shared script includes oversampling despite a historical description of v3 as “no rebalancing.” This unresolved source-history mismatch is a reason to rerun a controlled ablation, not to choose whichever account supports a preferred explanation.

Table 18: Every checkpoint evaluated in the functional test sweep. Development scores come from the separately preserved history.

Step	Development accuracy	Exposed-test accuracy
200	0.821	0.786
250	0.857	0.750
300	0.857	0.786
350	0.893	0.786
400	0.857	0.750
450	0.857	0.821
500	0.857	0.786
550	0.893	0.821
600	0.857	0.786

E Derivation of Aggregate Pairing Bounds

Let A and B be subsets of correctly answered rows, with $|A| = a$ and $|B| = b$ in a universe of size n . Since $|A \cup B| \leq n$, their intersection obeys $|A \cap B| \geq a + b - n$; nonnegativity and containment give the remaining bounds in Section 6.5. Every integer x in that range yields a feasible 2×2 correctness table:

$$\begin{array}{c|cc}
 & B \text{ correct} & B \text{ incorrect} \\
 \hline
 A \text{ correct} & x & a - x \\
 A \text{ incorrect} & b - x & n - a - b + x
 \end{array} . \tag{8}$$

At $x = 184$, the discordant counts are $(7, 0)$, giving $p = 2/2^7 = 0.015625$. At $x = 175$, they are $(16, 9)$, giving $p \approx 0.229523$. The released script enumerates all ten possibilities rather than assuming an overlap pattern. Unit tests compare the bounds against exhaustive binary correctness assignments on small universes. A paired F1 analysis would require the joint pattern of missing, incorrect, and correct field predictions, which these binary margins do not retain. Even complete vectors would still require attention to clustered listings and training contamination before inferential claims.

F Prospective Controlled Study

This appendix specifies experiments that would strengthen a future version; none is presented as already performed. The first gate is data integrity: create a new scenario bank, group splits before paraphrase generation, exclude item and normalized-text duplicates, and freeze a scored test set whose labels are not used by a training controller. Add blinded human judgments of label support and naturalness on a stratified sample, reporting annotator agreement and disagreement resolution separately from automated-judge agreement. Annotate whether the supplied evidence suffices to answer, and represent unknown cases explicitly.

The next gate is a matched baseline study. Evaluate the unfine-tuned base, shared adapter, equal-budget specialists, a supervised compact relevance classifier, Ditto-style identity matching, task-tuned extraction encoders, and relevant commerce models on the same test contracts. Apply model-specific prompt formatting selected only on development data. Preserve raw outputs, request IDs, exact provider/model revisions when available, all failed requests, parser versions, and token usage. Compute class macro-F1 and per-class recall, query-level metrics where the official search task requires them, and both exact and normalized field metrics. Use paired uncertainty estimates with query/entity/scenario grouping and disclose the family of comparisons.

For data-selection originality, test whether allocating a fixed annotation/token budget to new scenarios outperforms allocating it to additional paraphrases. Cross this with automated-judge acceptance versus blinded human filtering, keeping domain and label coverage matched. A learning curve over unique scenarios and total tokens would isolate diversity from repetition. Such a study could substantiate a data-selection contribution; the current scenario-overlap diagnosis alone does not establish a superior selection algorithm.

For training originality, compare uniform task sampling with a development-only controller that reacts to minority-task regression. Its intervention budget, stopping rule, and replay allocation must

be fixed before test access. Include existing balancing methods and equal-budget specialist controls; log per-task losses and gradient conflict if an interference mechanism is claimed. Three or more seeds would permit an initial variance estimate, but the final replication budget should depend on observed uncertainty and the effect size of interest. Test scores must never drive automatic retraining decisions.

For serving originality, compare separate model instances with one resident base and explicit adapter scheduling, then vary quantization, batch size, and concurrency while re-evaluating output quality. Record GPU model, memory, runtime versions, request lengths, cold/warm conditions, completion/error counts, and actual billed intervals. A scheduling method would need a throughput–tail-latency–quality advantage over existing serving strategies under the same workload. Measure on candidate hardware before choosing a deployment instance; the available logs cannot identify an optimal RunPod GPU or guarantee uninterrupted service.

A publishable systems or learning claim would require these interventions to be implemented, compared with relevant prior methods, and supported by independent test and serving evidence. The present paper instead offers a transparent empirical foundation and an executable account of which measurements remain missing.